

Estruturas de Dados: Questões Inéditas

JOÃO VÍCTOR BENETTI FILIPIM ,

JOÃO PEDRO MUSSI MEDEIROS

JOÃO PEDRO MUSSI MEDEIROS

MAURO PEZZETTA RONCATA

Universidade Regional do Noroeste do Estado do Rio Grande do Sul - Unijuí
Estruturas de Dados e Algoritmos
MARCOS RONALDO MELO CAVALHEIRO

O trabalho consiste na criação de questões inéditas com os conteúdos: busca linear, busca binária, Bubble Sort, Selection Sort, Insertion Sort e Quick-sort.



Questões Grupo 3

Questões Objetivas (7 questões)

Questão 1 — Objetiva (5 alternativas: A a E) Tema obrigatório: Busca linear: funcionamento e complexidade. **Enunciado:** Em um sistema de cadastro sem nenhuma organização específica (desordenado), um funcionário precisa encontrar o registro de um cliente pelo nome. Para isso, o sistema verifica os nomes um por um, do início ao fim da lista, até encontrar o cliente desejado ou a lista terminar. Como se chama esse algoritmo e qual é a sua complexidade de tempo no pior caso (quando o cliente é o último da lista ou não existe)?

a) Busca binária, com complexidade $O(\log n)$. b) Busca linear, com complexidade $O(n)$. c) Busca por saltos, com complexidade $O(1)$. d) Busca linear, com complexidade $O(n^2)$. e) Busca em profundidade, com complexidade $O(n \log n)$.

Gabarito (letra correta): B Justificativa (por que a correta está certa e as demais erradas):

A alternativa correta é a **B** porque a busca linear consiste exatamente em percorrer a lista sequencialmente. No pior caso, ela precisa checar todos os n elementos, resultando em complexidade $O(n)$. A letra A erra o nome e a complexidade; a letra C erra o nome e a complexidade; a letra D acerta o nome, mas erra a complexidade ($O(n^2)$ é para algoritmos elementares de ordenação); a letra E refere-se a algoritmos de grafos.

Questão 2 — Objetiva (5 alternativas: A a E) Tema obrigatório: Busca binária: funcionamento e complexidade. **Enunciado:** A busca binária é um algoritmo muito rápido utilizado para localizar a posição de um elemento em um vetor. Sobre a forma como a busca binária opera e a sua complexidade, assinale a alternativa correta:

a) Ela verifica cada elemento sequencialmente, garantindo o resultado em tempo $O(n)$. b) Ela sempre checa as extremidades do vetor antes do meio, operando em tempo $O(n^2)$. c) Ela divide sucessivamente a parte da lista que pode conter o elemento pela metade, possuindo complexidade de pior caso de $O(\log n)$. d) Ela divide a lista em três partes iguais a cada passo, com complexidade constante $O(1)$. e) Ela ordena o vetor enquanto busca, com complexidade $O(n \log n)$.

Gabarito (letra correta): C Justificativa (por que a correta está certa e as demais erradas):

A correta é a **C** porque descreve com exatidão o princípio da "divisão e conquista" da busca binária, que corta o espaço de busca pela metade a cada iteração, alcançando tempo logarítmico ($O(\log n)$). A letra A descreve a busca linear. A letra B inventa uma mecânica inexistente. A letra D propõe tempo constante, o que é impossível sem saber a posição exata (como em tabelas Hash). A letra E mistura conceitos de ordenação com busca.

Questão 3 — Objetiva (5 alternativas: A a E) Tema obrigatório: Busca binária: pré-condição de ordenação e consequências de aplicar o algoritmo a dados não ordenados. **Enunciado:** Um estudante de programação aprendeu que a busca binária é mais rápida que a busca linear e decidiu aplicá-la em todos os vetores numéricos de seu código. Porém, ele tentou buscar o número 15 em um vetor desordenado: [20, 5, 15, 30, 2]. O que acontece quando aplicamos a busca binária em dados não ordenados?

a) O algoritmo ordena o vetor automaticamente antes de começar a procurar o número 15. b) O algoritmo vai encontrar o número 15 normalmente, mas funcionará mais lentamente, como uma busca linear. c) O algoritmo acusará um erro de sintaxe e o programa não compilará. d) O comportamento é imprevisível e o algoritmo pode não encontrar o número 15, retornando que ele não existe (falso negativo). e) O algoritmo entra em loop infinito.

Gabarito (letra correta): D Justificativa (por que a correta está certa e as demais erradas):

A alternativa **D** está correta. A pré-condição fundamental da busca binária é que a lista esteja ordenada. Se o vetor está desordenado, a decisão de ir para a metade direita ou esquerda com base no elemento central levará a caminhos errados, causando falhas na busca. A letra A é falsa (o algoritmo não ordena sozinho). A letra B é falsa (o algoritmo quebra sua lógica, não fica apenas lento). As letras C e E são falsas (não dá erro de compilação nem loop infinito garantido, ele apenas encerra sem achar o dado).

Questão 4 — Objetiva (5 alternativas: A a E) Tema obrigatório: Bubble Sort: funcionamento, trocas e análise de casos. **Enunciado:** O Bubble Sort (Ordenação Bolha) funciona comparando elementos adjacentes e os trocando de lugar se estiverem na ordem errada. Imagine que ele receba um vetor que **já está completamente ordenado** (ex: [1, 2, 3, 4, 5]). Em uma versão otimizada do algoritmo, o que ocorrerá neste cenário de melhor caso?

a) Ele fará várias trocas sem necessidade e terá complexidade $O(n^2)$. b) Ele não realizará nenhuma troca e, ao perceber que não houve trocas na primeira passagem, encerrará com complexidade $O(n)$. c) Ele dividirá o vetor ao meio para confirmar a ordenação, com complexidade $O(\log n)$. d) Ele realizará trocas mesmo assim, mas terminará em tempo $O(1)$. e) Ele apagará os dados duplicados e ordenará com tempo $O(n \log n)$.

Gabarito (letra correta): B Justificativa (por que a correta está certa e as demais erradas):

A correta é a **B**. O Bubble Sort otimizado utiliza uma variável (flag) para verificar se houve alguma troca durante uma passada no vetor. Se não houver trocas na primeira iteração (porque já está ordenado), ele entende que a lista está pronta e para, resultando em complexidade $O(n)$. As outras alternativas citam complexidades ou comportamentos incorretos para o melhor caso do Bubble Sort.

Questão 5 — Objetiva (5 alternativas: A a E) Tema obrigatório: Selection Sort: funcionamento, seleção do menor/maior elemento e análise de casos. **Enunciado:** O Selection

Sort (Ordenação por Seleção) constrói a lista final ordenada encontrando repetidamente o menor (ou maior) elemento da parte não ordenada e colocando-o na posição correta. Sobre este algoritmo, é correto afirmar:

- a) Ele é o melhor algoritmo de ordenação para listas enormes, superando o Quick-sort.
- b) Sua complexidade é $O(n)$ no melhor caso, quando a lista já está ordenada.
- c) Ele realiza o mesmo número de comparações no melhor, médio e pior caso, resultando sempre em uma complexidade de tempo de $O(n^2)$.
- d) Ele não realiza trocas de elementos em momento algum de sua execução.
- e) Ele só funciona com números inteiros positivos.

Gabarito (letra correta): C Justificativa (por que a correta está certa e as demais erradas):

A letra **C** é a correta. O Selection Sort não se adapta aos dados; ele sempre precisa varrer todo o restante do vetor para garantir que encontrou o menor elemento daquela iteração, resultando em $O(n^2)$ comparações independentemente do estado inicial do vetor. As letras **A** e **B** estão incorretas pois ele é ineficiente ($O(n^2)$ sempre). A letra **D** é incorreta porque o algoritmo faz trocas ao final de cada seleção. A letra **E** é falsa, ele ordena qualquer tipo de dado comparável.

Questão 6 — Objetiva (5 alternativas: A a E) Tema obrigatório: Insertion Sort:

funcionamento, comportamento em dados parcialmente ordenados e análise de casos.

Enunciado: O Insertion Sort (Ordenação por Inserção) é muito comparado à maneira como organizamos cartas de baralho nas mãos. Ele possui uma característica muito vantajosa quando lida com um vetor que está **quase totalmente ordenado**. Qual é essa característica?

- a) Ele se comporta muito mal, atingindo sempre seu pior caso $O(n^2)$ com dados parcialmente ordenados.
- b) Ele consegue ordenar de forma instantânea, com complexidade de $O(1)$.
- c) Ele realiza poucas movimentações e sua complexidade de tempo cai significativamente, aproximando-se de seu melhor caso que é $O(n)$.
- d) Ele aciona automaticamente a busca binária para embaralhar as cartas de novo.
- e) Ele exige o dobro de memória RAM para processar dados parcialmente ordenados.

Gabarito (letra correta): C Justificativa (por que a correta está certa e as demais erradas):

A correta é a **C**. O Insertion Sort insere o elemento atual na porção já ordenada à esquerda. Se a lista estiver quase ordenada, ele fará poucos deslocamentos, pois o elemento já estará muito próximo (ou na) sua posição correta, operando em tempo muito próximo de $O(n)$. A letra **A** inverte a lógica (esse é o forte dele). A letra **B** propõe $O(1)$, o que é impossível. As letras **D** e **E** contêm afirmações fantasiosas.

Questão 7 — Objetiva (5 alternativas: A a E) Tema obrigatório: Quick-sort: particionamento,

escolha do pivô, estabilidade e/ou análise de complexidade. **Enunciado:** O algoritmo Quick-sort escolhe um elemento chamado "pivô" e particiona o vetor de modo que os menores fiquem de um lado e os maiores de outro. No entanto, o Quick-sort pode sofrer uma queda brusca de

desempenho (chegando à complexidade $O(n^2)$) em uma situação específica de pior caso. Qual alternativa descreve corretamente esse pior caso?

a) Quando a escolha do pivô é sempre o maior ou o menor elemento em um vetor que já está ordenado (ou ordenado reversamente). b) Quando o vetor contém apenas números primos. c) Quando o pivô escolhido é sempre a exata mediana dos dados. d) Quando o vetor tem tamanho ímpar, impossibilitando a divisão. e) Quando o particionamento é feito de forma aleatória, dividindo sempre o vetor exatamente na metade.

Gabarito (letra correta): A Justificativa (por que a correta está certa e as demais erradas):

A alternativa A está correta. O pior caso do Quick-sort ($O(n^2)$) ocorre quando as partições geradas são extremamente desbalanceadas (uma partição com $n-1$ elementos e outra com 0). Isso geralmente ocorre quando o pivô escolhido é o elemento nas extremidades (maior ou menor) de uma lista que já está ordenada ou em ordem reversa. A alternativa C e E descrevem exatamente os cenários de *melhor* caso ($O(n \log n)$). Letras B e D são irrelevantes para a complexidade.

Questões Dissertativas/Práticas (3 questões)

Questão 8 — Dissertativa/Prática Tema obrigatório: Comparar busca linear e busca binária, considerando pré-condições, número de comparações e complexidade. **Enunciado:** Uma empresa tem um catálogo de 1.000 produtos e precisa criar uma função para buscar um produto pelo seu código. Responda:

1. Qual a pré-condição necessária para que a Busca Binária possa ser utilizada nesse catálogo?
2. Compare a Busca Linear e a Busca Binária em termos de eficiência (complexidade) e estime quantas comparações (no pior caso) cada uma fará para encontrar um produto.

Resolução comentada / Gabarito / Critérios de correção:

- **Critério 1:** O aluno deve responder que a pré-condição para utilizar a Busca Binária é que os códigos dos produtos estejam **ordenados**.
- **Critério 2:** Deve explicar que a Busca Linear possui complexidade $O(n)$ e a Busca Binária $O(\log n)$.
- **Critério 3 (estimativa de comparações):** No pior caso, a Busca Linear fará até 1.000 comparações (verificará todos os elementos). A Busca Binária fará, no máximo, em torno de 10 comparações (pois $\log_2(1000) \approx 9,96$, arredondando para 10 divisões).

Questão 9 — Dissertativa/Prática Tema obrigatório: Comparar Bubble Sort, Selection Sort e Insertion Sort, incluindo estabilidade e melhor caso, caso médio e pior caso quando aplicável.

Enunciado: Você recebeu uma lista de alunos de uma escola que foi recém-atualizada: ela já estava organizada em ordem alfabética, mas três novos alunos foram adicionados no final e a

lista precisa ser reordenada. Entre os algoritmos Bubble Sort (otimizado), Selection Sort e Insertion Sort, qual deles você escolheria para esta tarefa específica de ordenar um vetor "quase totalmente ordenado"? Justifique sua resposta comparando a complexidade de melhor caso dos três.

Resolução comentada / Gabarito / Critérios de correção:

- **Escolha:** O aluno deve escolher o **Insertion Sort** (ou o Bubble Sort otimizado, que também é aceitável, mas o Insertion é classicamente o ideal para isso).
- **Justificativa / Comparação:**
 - O **Selection Sort** deve ser descartado, pois não detecta a ordem pré-existente e fará varreduras redundantes, sempre operando em $O(n^2)$ (melhor, médio e pior caso).
 - O **Insertion Sort** possui melhor caso $O(n)$. Como os primeiros elementos já estão ordenados, o algoritmo não fará movimentações nas primeiras verificações, apenas pegará os três últimos nomes e os "encaixará" rapidamente em suas posições.
 - O **Bubble Sort (otimizado)** também teria bom desempenho ($O(n)$), mas requer mais comparações sequenciais desnecessárias em relação ao Insertion Sort para movimentar elementos recém adicionados ao fim da lista para o início/meio.

Questão 10 — Dissertativa/Prática Tema obrigatório: Analisar o Quick-sort: particionamento, escolha do pivô, estabilidade, melhor caso, caso médio e pior caso. **Enunciado:** O Quick-sort é um algoritmo eficiente baseado em "dividir para conquistar". A) Explique rapidamente como funciona a fase de particionamento em relação ao pivô. B) O que determina se o Quick-sort vai rodar na sua eficiência de caso médio ($O(n \log n)$) ou cair no seu pior caso terrível ($O(n^2)$)? Cite uma estratégia para evitar o pior caso.

Resolução comentada / Gabarito / Critérios de correção:

- **Critério A (Particionamento):** O aluno deve explicar que um elemento é escolhido como pivô. Durante o particionamento, os elementos do vetor são reorganizados de forma que todos os valores menores que o pivô fiquem à sua esquerda, e os maiores à sua direita. Com isso, o pivô assume sua posição definitiva.
- **Critério B (Complexidade e Estratégia):** O que determina o desempenho é a qualidade das divisões. Divisões ao meio garantem o tempo logarítmico ($O(n \log n)$). O pior caso ocorre quando as divisões são extremamente desbalanceadas (ex: quando a lista está ordenada e o pivô escolhido é o primeiro/último elemento). Para evitar isso, o aluno deve citar estratégias como:
 - Escolher o pivô de forma aleatória.
 - Utilizar a técnica "Mediana de 3" (escolher o pivô como o valor do meio entre o primeiro, o central e o último elemento do vetor).

Declaração de Uso de Inteligência Artificial

Conforme as regras de preenchimento, abaixo está o quadro que descreve o uso da IA nesta exata solicitação de geração de conteúdo:

Declaração do grupo: *Ferramenta utilizada:* ChatGPT / Gemini / IA Generativa. *Finalidade:* Geração de ideias e elaboração da estrutura base das 10 questões, mantendo-as em nível de dificuldade introdutório e didático.

Etapas:

- 1) Inserção do template desejado.
- 2) Especificação da dificuldade.
- 3) Geração do texto das questões.

Procedimentos de verificação: Leitura e validação dos gabaritos de tempo de complexidade (Big-O) gerados para assegurar o rigor técnico e teórico da Ciência da Computação. *Fontes de validação:* Literatura clássica de Algoritmos e Estruturas de Dados (ex: Thomas Cormen, Nivio Ziviani).